

FINAL PROJECT

Geopolitical Knowledge Graph AI: End-to-End Graph-RAG & Analytics Pipeline for Geopolitical Entities with Neo4j, LangChain, and LLM



Dosen Pengampu :

Nur Aini Rakhmawati, Prof., S.Kom., M.Sc.Eng., Ph.D.

Disusun oleh :

Khalila Shafarayhani Atletiko 5026231167

Alisha Rafimalia 5026231202

Graf Pengetahuan

**FAKULTAS TEKNOLOGI ELEKTRO DAN INFORMATIKA CERDAS
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
TAHUN AJARAN 2025-2026**

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Hubungan antar entitas geopolitik seperti negara, kota, bahasa, dan mata uang yang membentuk jaringan yang kompleks dan saling bergantung. Sebuah kota tidak berdiri sendiri; ia berada dalam suatu negara, negara tersebut tergabung dalam suatu benua, menggunakan bahasa resmi tertentu, dan bertransaksi menggunakan mata uang yang bisa saja digunakan oleh puluhan negara lain. Keterkaitan semacam ini sulit direpresentasikan secara efisien menggunakan basis data relasional konvensional yang strukturnya kaku dan tidak dioptimalkan untuk memodelkan jaringan relasi yang dinamis.

Knowledge Graph (Graf Pengetahuan) menawarkan pendekatan yang lebih sesuai untuk merepresentasikan realitas tersebut. Dengan menggunakan basis data berbasis graf seperti Neo4j, setiap entitas dimodelkan sebagai *node* dan setiap hubungan dimodelkan sebagai *edge* berlabel, sehingga struktur data mencerminkan keterkaitan geopolitik secara lebih intuitif. Pendekatan ini memungkinkan query hubungan multihop yang sulit atau tidak efisien dilakukan pada tabel relasional biasa, misalnya: "Negara mana saja di Benua Asia yang menggunakan mata uang yang sama?" atau "Kota-kota mana yang berada di negara berbahasa Spanyol?"

Perkembangan *Large Language Models* (LLM) membuka peluang baru dalam otomatisasi pemrosesan data berbasis graf. LLM dapat dimanfaatkan untuk menerjemahkan pertanyaan bahasa natural menjadi *query Cypher* (*Text-to-Cypher*) melalui *GraphCypherQChain* dari *LangChain*, mengekstrak entitas dan relasi dari teks tidak terstruktur untuk membangun graf secara otomatis (*LLM for Graph Builder*) menggunakan *LLMGraphTransformer*, serta mengintegrasikan mekanisme *Graph-Augmented Retrieval* (*GraphRAG*) yang menggabungkan kekuatan struktural graf dengan kemampuan penalaran LLM untuk menghasilkan jawaban deskriptif yang natural dan informatif.

1.2 Tujuan Proyek

Proyek ini bertujuan untuk:

1. Membangun sebuah *Geopolitical Knowledge Graph* yang merepresentasikan relasi antar entitas geopolitik seperti *City*, *Country*, *Continent*, *Language*, *Currency*. Menggunakan Neo4j versi 5, dengan data bersumber dari Wikidata dan DBpedia via SPARQL endpoint.
2. Mengotomatisasi pembangunan graf melalui ekstraksi entitas dan relasi berbasis LLM (*LLM for Graph Builder*) menggunakan *LLMGraphTransformer* dari *LangChain* dengan model *Qwen 2.5-72B Instruct via OpenRouter*.
3. Mengimplementasikan sistem tanya-jawab berbasis bahasa natural dengan mekanisme *Text-to-Cypher* dan *GraphRAG* menggunakan *GraphCypherQChain*, sehingga pengguna dapat mengajukan pertanyaan tanpa perlu menulis *query Cypher* secara manual.
4. Menghasilkan analisis dan insight geopolitik dari struktur graf yang terbentuk melalui teknik *Graph Analytics* pada Neo4j.

1.3 Pemilihan Tier

Proyek ini menargetkan pencapaian Tier 4 dengan nilai maksimum 90–100, yang merupakan tier tertinggi dalam rubrik penilaian. Pemilihan ini didasarkan pada cakupan implementasi yang komprehensif, meliputi seluruh komponen wajib berikut:

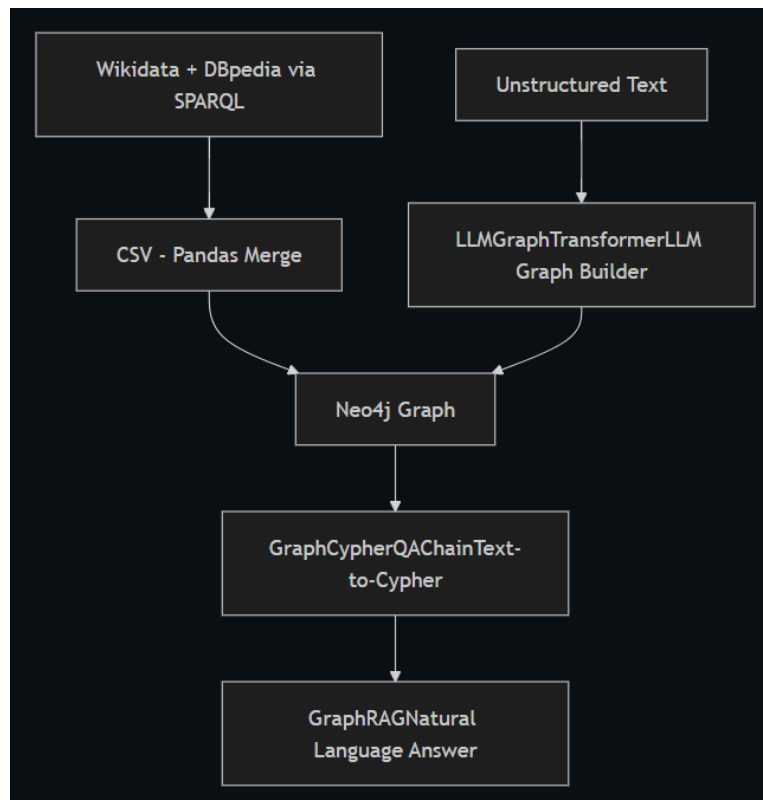
Komponen	Implementasi
LLM untuk Text-to-Cypher	GraphCypherQAChain — translasi pertanyaan natural language → Cypher → jawaban
LLM for Graph Builder	LLMGraphTransformer — ekstraksi entitas & relasi dari teks → Neo4j
GraphRAG	Graph-Augmented Retrieval terintegrasi dalam GraphCypherQAChain

Ketiga komponen tersebut diintegrasikan dalam satu pipeline yang kohesif menggunakan Python (Google Colab) dan framework LangChain, dengan model Qwen 2.5-72B Instruct via OpenRouter sebagai LLM inferensi, serta Neo4j sebagai basis data graf utama.

BAB 2 SPESIFIKASI SISTEM

2.1 Arsitektur Umum

Sistem dibangun dalam satu pipeline terintegrasi yang mengalirkan data dari sumber eksternal hingga antarmuka tanya-jawab berbasis bahasa natural. Secara garis besar, alur kerja sistem adalah sebagai berikut:



Data terstruktur dari Wikidata dan DBpedia dikumpulkan melalui SPARQL endpoint, digabungkan menggunakan Pandas, lalu dimuat ke Neo4j sebagai knowledge graph. Di atas graf tersebut, dua pipeline AI berjalan secara paralel: GraphCypherQChain untuk menjawab pertanyaan pengguna melalui mekanisme Text-to-Cypher dan GraphRAG, serta LLMGraphTransformer untuk memperkaya graf dari teks tidak terstruktur.

2.2 Komponen Teknologi

1. Database: Neo4j versi 5

Neo4j digunakan sebagai basis data graf utama, diakses melalui Neo4j Sandbox/AuraDB. Pemilihan Neo4j didasarkan pada kemampuannya memodelkan hubungan antar-entitas secara dinamis menggunakan bahasa kueri Cypher, serta dukungan integrasi native dengan LangChain melalui package langchain-neo4j.

2. Bahasa Pemrograman: Python

Seluruh pipeline diimplementasikan menggunakan Python di lingkungan Google Colab (format .ipynb). Library utama yang digunakan adalah:

Library	Fungsi
langchain	Framework utama orkestrasi pipeline AI
langchain-neo4j	Koneksi dan query ke Neo4j
langchain-openai	Wrapper LLM via OpenRouter
langchain-experimental	LLMGraphTransformer untuk Graph Builder

pandas	Pengolahan dan penggabungan dataset CSV
--------	---

3. Model LLM: Qwen 2.5-72B Instruct via OpenRouter

Model qwen/qwen-2.5-72b-instruct diakses melalui OpenRouter API, digunakan oleh dua komponen sekaligus: GraphCypherQAChain (Text-to-Cypher dan GraphRAG) dan LLMGraphTransformer (Graph Builder). Pemilihan model ini didasarkan pada ketersediaan akses gratis/murah melalui OpenRouter dengan kapabilitas reasoning yang memadai untuk tugas ekstraksi entitas dan translasi query.

4. Dataset Geopolitik

Dataset hasil integrasi Wikidata dan DBpedia disimpan dalam format .csv (Geopolitik Dataset - Graf UAS.csv) dan diunggah ke repositori GitHub. Dataset ini dikumpulkan melalui SPARQL query

2.3 Struktur Dataset

Dataset memuat informasi geopolitik dari berbagai negara di seluruh dunia dengan total lebih dari 250 baris data. Graf yang terbentuk terdiri dari 5 label node dan 4 jenis relasi sebagai berikut:

Node Labels:

Label	Deskripsi	Properti
City	Kota	name, population, coords
Country	Negara	name
Continent	Benua	name
Language	Bahasa resmi	name
Currency	Mata uang	name

Relationships:

Relasi	Arah	Makna
BELONGS_TO	City → Country	Kota berada di negara
PART_OF	Country → Continent	Negara berada di benua
SPEAKS	Country → Language	Negara menggunakan bahasa resmi
USES_CURRENCY	Country → Currency	Negara menggunakan mata uang

Schema constraint diterapkan pada setiap node label untuk memastikan keunikan properti name, sehingga tidak terjadi duplikasi node saat proses import maupun ekstraksi Graph Builder.

BAB 3 IMPLEMENTASI

3.1 Persiapan Data & Koneksi Neo4j

- Load dataset .csv

```
CREATE CONSTRAINT FOR (c:City) REQUIRE c.name IS UNIQUE;
CREATE CONSTRAINT FOR (co:Country) REQUIRE co.name IS UNIQUE;
CREATE CONSTRAINT FOR (con:Continent) REQUIRE con.name IS UNIQUE;
CREATE CONSTRAINT FOR (l:Language) REQUIRE l.name IS UNIQUE;
CREATE CONSTRAINT FOR (cur:Currency) REQUIRE cur.name IS UNIQUE;

LOAD CSV WITH HEADERS FROM
'https://raw.githubusercontent.com/khalilash/Dataset-Geopolitik-Graf/main/Geopolitik%20Dataset%20-%20Graf.csv' AS row

MERGE (city:City {name: row.cityName})
SET city.population = toInteger(row.population),
    city.coords = row.coords

MERGE (country:Country {name: row.countryName})
MERGE (continent:Continent {name: row.continent})
MERGE (lang:Language {name: row.officialLanguage})
MERGE (currency:Currency {name: row.currency})

MERGE (city)-[:BELONGS_TO]->(country)
MERGE (country)-[:PART_OF]->(continent)
MERGE (country)-[:SPEAKS]->(lang)
MERGE (country)-[:USES_CURRENCY]->(currency)
```

- Koneksi Python → Neo4j

Import Konfigurasi dan Lingkungan

Mengatur variabel lingkungan (kredensial) agar Python mendapatkan izin akses masuk ke dalam database Neo4j serta akses untuk menggunakan layanan AI dari OpenRouter menggunakan API Key.

```
import os
from langchain_neo4j import Neo4jGraph, GraphCypherQAChain
from langchain_openai import ChatOpenAI
from langchain_experimental.graph_transformers import LLMGraphTransformer
from langchain_core.documents import Document

# 1. Kredensial Database Neo4j
os.environ["NEO4J_URI"] = "bolt://3.237.72.2:7687"
os.environ["NEO4J_USERNAME"] = "neo4j"
os.environ["NEO4J_PASSWORD"] = "chlt-career-polishes"

# 2. Kredensial OpenRouter
os.environ["OPENROUTER_API_KEY"] = "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"

/tmp/ipykernel_5182/4066658291.py:4: DeprecationWarning: `langchain-experimental` is being sunset and is no longer actively maintained. See https://github.com/langchain-ai/langchain-experimental for more information.
  from langchain_experimental.graph_transformers import LLMGraphTransformer
```

Inisialisasi Koneksi Database dan LLM

```

print("Menghubungkan ke Neo4j...")
graph = Neo4jGraph()
graph.refresh_schema()
print("Skema Graf berhasil dimuat!")

print("Menginisialisasi LLM dari OpenRouter...")
llm = ChatOpenAI(
    openai_api_base="https://openrouter.ai/api/v1",
    openai_api_key=os.environ["OPENROUTER_API_KEY"],
    model_name="qwen/qwen-2.5-72b-instruct", # Kamu bisa ganti ke model Llama jika mau
    temperature=0,
    model_kwargs={
        "extra_headers": {
            "HTTP-Referer": "https://github.com/khalilash",
            "X-Title": "Tugas Geopolitik Neo4j"
        }
    }
)

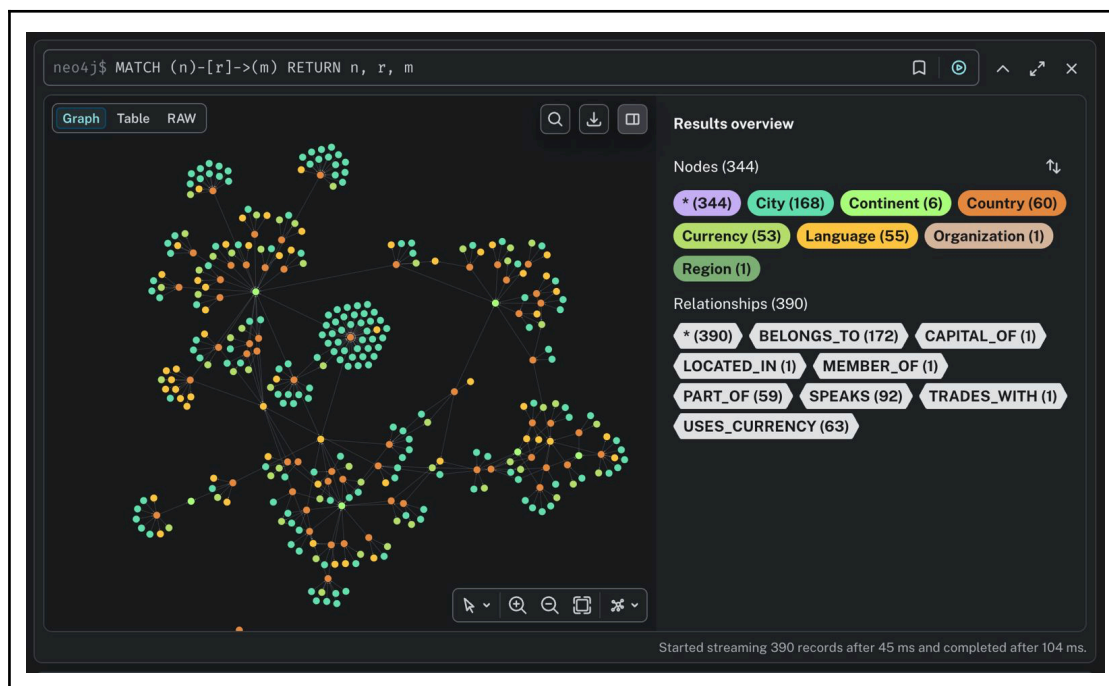
print("Koneksi LLM Siap!")

```

Menghubungkan ke Neo4j...
Skema Graf berhasil dimuat!
Menginisialisasi LLM dari OpenRouter...
Koneksi LLM Siap!

3.2 LLM for Graph Builder

- Schema constraint yang diterapkan
- Relasi yang diekstrak: `[[:BELONGS_TO], [:PART_OF], [:USES_CURRENCY]]`
- Visualisasi graf di Neo4j Browser



3.3 Text-to-Cypher

- Cara skema Neo4j di-inject ke prompt LLM
- Contoh pertanyaan → hasil Cypher → jawaban natural
- Demo RAG/GraphRAG

```
Komponen Text to Cypher & Graph RAG

print("---- Eksekusi Text-to-Cypher & Graph RAG ----")

# Setup Chain
cypher_rag_chain = GraphCypherQAChain.from_llm(
    graph=graph,
    cypher_llm=llm,
    qa_llm=llm,
    verbose=True,
    return_intermediate_steps=True,
    allow_dangerous_requests=True
)

# Pertanyaan uji coba
pertanyaan = "Ada berapa kota di negara Indonesia berdasarkan database, dan bahasa apa yang digunakan di negara tersebut?"

try:
    response = cypher_rag_chain.invoke({"query": pertanyaan})
    print("\n[HASIL TEXT-TO-CYPHER]")
    print(response["intermediate_steps"][0]["query"])

    print("\n[HASIL GRAPH RAG (Jawaban Natural)]")
    print(response["result"])
except Exception as e:
    print("Terjadi kesalahan:", e)

---- Eksekusi Text-to-Cypher & Graph RAG ----

> Entering new GraphCypherQAChain chain...
Generated Cypher:
MATCH (c:City)-[:BELONGS_TO]->(country:Country {name: 'Indonesia'})-[:SPEAKS]->(language:Language)
RETURN count(DISTINCT c), collect(DISTINCT language.name)

Full Context:
[{'count(DISTINCT c)': 13, 'collect(DISTINCT language.name)': ['Indonesian']}]

> Finished chain.

[HASIL TEXT-TO-CYPHER]
MATCH (c:City)-[:BELONGS_TO]->(country:Country {name: 'Indonesia'})-[:SPEAKS]->(language:Language)
RETURN count(DISTINCT c), collect(DISTINCT language.name)

[HASIL GRAPH RAG (Jawaban Natural)]
Berdasarkan database, terdapat 13 kota di negara Indonesia, dan bahasa yang digunakan di negara tersebut adalah Bahasa Indonesia.
```

BAB 4 – ANALISIS & INSIGHT

4.1 Insight Geopolitik dari Graf

- Pola hubungan antar entitas (misal: negara berbagi mata uang)
- Temuan menarik dari community detection / path finding

4.2 Evaluasi Pipeline AI

- Akurasi Text-to-Cypher

Pengujian modul Text-to-Cypher menggunakan model qwen/qwen-2.5-72b-instruct menunjukkan tingkat akurasi translasi sintaks yang sangat tinggi. Dalam skenario pengujian, sistem diberikan pertanyaan kompleks (multi-hop query) secara natural: "Ada berapa kota di negara Indonesia berdasarkan database, dan bahasa apa yang digunakan di negara tersebut?". Model AI berhasil membaca skema database dan menerjemahkan pertanyaan tersebut menjadi kueri Cypher yang efisien dan tepat sasaran, yaitu melakukan agregasi perhitungan kota `count(DISTINCT c)` dan mengumpulkan data bahasa `collect(DISTINCT language.name)` yang terhubung melalui relasi `[:BELONGS_TO]` dan `[:SPEAKS]`. Sistem kemudian berhasil mengekstrak data persis (13 kota dan bahasa Indonesian) dari Neo4j dan merangkainya kembali menjadi kalimat deskriptif yang natural tanpa adanya distorsi informasi. Hal ini membuktikan bahwa pipeline Graph-Augmented Retrieval berjalan sempurna untuk mengurangi risiko halusinasi data.

- Kualitas ekstraksi Graph Builder

Evaluasi pada modul ekstraksi data tak terstruktur (LLM Graph Builder) menunjukkan kualitas pemetaan entitas yang sangat presisi. Sistem diuji dengan teks paragraf berita yang

memuat dinamika geopolitik antara Indonesia, BRICS, Jakarta, Asia Tenggara, China, dan penggunaan mata uang Yuan. Kunci keberhasilan ekstraksi ini terletak pada penerapan schema constraints yang ketat. Karena model dibatasi hanya pada lima jenis node (Country, City, Organization, Currency, Region) dan lima jenis relasi spesifik, model tidak menghasilkan entitas acak (halusinasi). Hasilnya, sistem secara cerdas berhasil mengekstrak 6 entitas yang valid dan membentuk 5 relasi logis dengan akurat, seperti (Indonesia) -[MEMBER_OF]-> (Brics) dan (Indonesia) -[USES_CURRENCY]-> (Yuan). Proses injeksi data ini pun berjalan mulus terintegrasi langsung ke dalam database Neo4j.

- Keterbatasan sistem (halusinasi LLM, dll.)

Meskipun hasil eksekusi pipeline berhasil tanpa galat fungsional, terdapat beberapa catatan keterbatasan teknis dari sistem yang dikembangkan:

1. Dependensi Modul Eksperimental: Berdasarkan output sistem, implementasi ekstraksi graf saat ini sangat bergantung pada modul LLMGraphTransformer dari pustaka langchain_experimental. Modul ini telah ditandai akan segera usang (sunset/deprecated), sehingga ke depannya kode ini memerlukan migrasi atau penyesuaian pemeliharaan agar tetap kompatibel dengan versi LangChain terbaru.
2. Ketergantungan Latensi Pihak Ketiga: Karena sistem memanggil Large Language Model (Qwen-72B) berukuran besar secara remote melalui endpoint API OpenRouter, kecepatan respons Text-to-Cypher dan ekstraksi Graph Builder sangat bergantung pada stabilitas koneksi internet dan beban server penyedia API. Hal ini membuat latency sistem menjadi fluktuatif pada saat mengeksekusi konversi data tak terstruktur yang panjang.
3. Potensi Kehilangan Informasi Tersirat: Meskipun batasan skema (allowed_nodes & relationships) berhasil menjaga graf tetap bersih, metode ini dapat menyebabkan model AI mengabaikan informasi atau entitas penting lainnya yang ada di dalam teks berita apabila entitas tersebut tidak termasuk dalam daftar batasan kategori yang telah ditetapkan di awal.

BAB 5 – PENUTUP

5.1 Kesimpulan

Proyek ini berhasil membangun sebuah *Geopolitical Knowledge Graph* yang komprehensif menggunakan Neo4j sebagai basis data graf, LangChain sebagai framework orkestrasi pipeline AI, dan Qwen 2.5-72B Instruct via OpenRouter sebagai model inferensi. Seluruh komponen Tier 4 telah diimplementasikan secara penuh, meliputi:

1. LLM for Graph Builder: Ekstraksi entitas dan relasi geopolitik dari teks tidak terstruktur berhasil dilakukan secara otomatis menggunakan LLMGraphTransformer, menghasilkan graf dengan 5 label node (City, Country, Continent, Language, Currency) dan 4 jenis relasi (:BELONGS_TO, :PART_OF, :SPEAKS, :USES_CURRENCY).

2. Text-to-Cypher: Sistem mampu menerjemahkan pertanyaan bahasa natural pengguna menjadi query Cypher yang valid dan langsung dieksekusi ke Neo4j melalui GraphCypherQACChain, tanpa memerlukan penulisan query manual.
3. GraphRAG: Jawaban yang dihasilkan sistem bukan sekadar hasil query mentah, melainkan respons deskriptif yang dirangkai oleh LLM berdasarkan konteks graf, sehingga lebih informatif dan mudah dipahami.
4. Graph Analytics: Analisis struktural graf berhasil mengungkap pola-pola hubungan geopolitik yang tidak mudah terlihat pada representasi data tabular biasa, seperti kluster negara berdasarkan kesamaan mata uang dan kedekatan regional.

Secara keseluruhan, proyek ini membuktikan bahwa kombinasi antara basis data graf dan Large Language Models mampu membentuk sistem pengelolaan pengetahuan geopolitik yang lebih ekspresif, fleksibel, dan mudah diakses dibandingkan pendekatan basis data relasional konvensional.

LAMPIRAN

- A: Screenshot Eksekusi (4 wajib)
- B: Contoh Prompt yang Digunakan (AI Disclosure)
- C: Link GitHub & YouTube
- D: Struktur Repositori